Date: 25/03/2026

EX-09

Deployment of Honeypots and Analysis of Botnet Activities

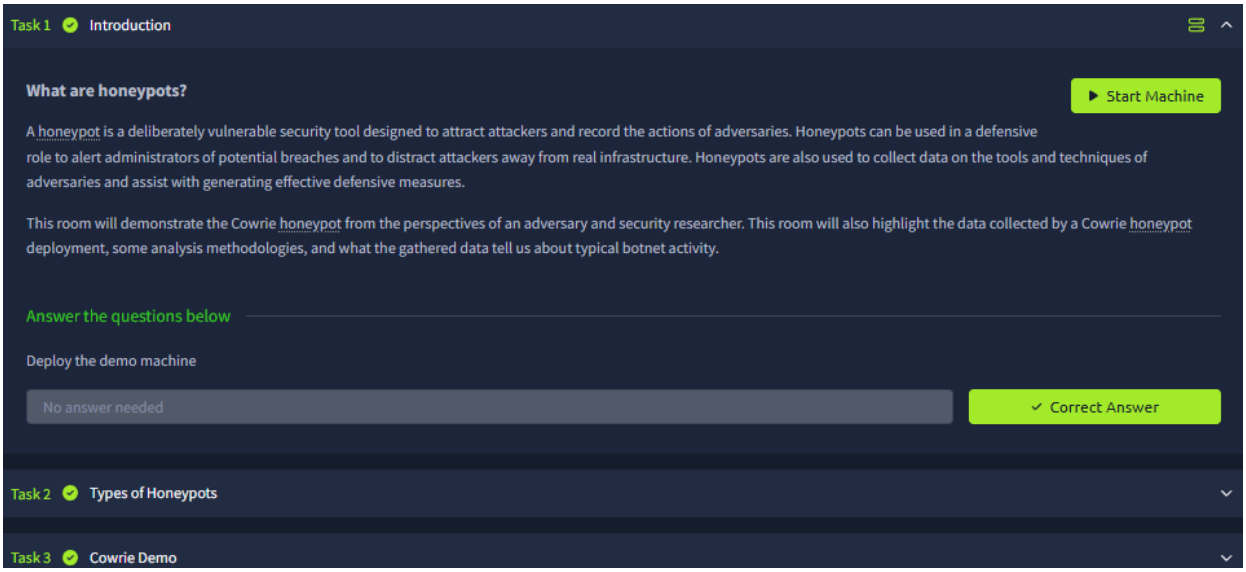
AIM:

To understand the deployment of honeypots and analyze botnet activities by capturing, logging, and studying attacker interactions in a controlled environment.

PROCEDURE:

- Set up a Cowrie honeypot on a system to simulate vulnerable SSH services.
- Configure logging and monitoring tools to capture all incoming attacker traffic.
- Allow the honeypot to collect data from potential attackers and bots.
- Analyze logs to identify patterns such as repeated login attempts, commands, and source IPs.
- Study the behavior of botnets and extract indicators of compromise (IoCs) from collected data.

TASK-1:



TASK-2:

Task 2 ✓ Types of Honeypots

Honeypot Interactivity and Classification

A wide variety of honeypots exist so it is helpful to classify them by the level of interactivity provided to adversaries, with most honeypots falling into one of the below categories:

- **Low-Interaction** honeypots offer little interactivity to the adversary and are only capable of simulating the functions that are required to simulate a service and capture attacks against it. Adversaries are not able to perform any post-exploitation activity against these honeypots as they are unable to fully exploit the simulated service. Examples of low-interaction honeypots include mailoney and dionaea.
- **Medium-Interaction** honeypots collect data by emulating both vulnerable services as well as the underlying OS, shell, and file systems. This allows adversaries to complete initial exploits and carry out post-exploitation activity. Note, that unlike, High-Interaction honeypots (see below), the system presented to adversaries is a simulation. As a result, it is usually not possible for adversaries to complete their full range of post-exploitation activity as the simulation will be unable to function completely or accurately. We will be taking a look at the medium-interaction SSH honeypot, Cowrie in this demo.
- **High-Interaction** honeypots are fully complete systems that are usually Virtual Machines that include deliberate vulnerabilities. Adversaries should be able (but not necessarily allowed) to perform any action against the honeypot as it is a complete system. It is important that high-interaction honeypots are carefully managed, otherwise, there is a risk that an adversary could use the honeypot as a foothold to attack other resources. Cowrie can also operate as an SSH proxy and management system for high-interaction honeypots.

Deployment Location

Once deployed, honeypots can then be further categorized by the exact location of their deployment:

- **Internal honeypots** are deployed inside a LAN. This type can act as a way to monitor a network for threats originating from the inside, for example, attacks originating from trusted personnel or attacks that by-parse firewalls like phishing attacks. Ideally, these honeypots should never be compromised as this would indicate a significant breach.
- **External honeypots** are deployed on the open internet and are used to monitor attacks from outside of the LAN. These honeypots are able to collect much more data on attacks since they are effectively guaranteed to be under attack at all times.

Answer the questions below

Read and understand the above

No answer needed | ✓ Correct Answer

TASK-3:

Task 3 ✓ Cowrie Demo

The Cowrie SSH Honeypot

The Cowrie honeypot can work both as an SSH proxy or as a simulated shell. The demo machine is running the simulated shell. You can log in using the following credentials:

1. IP - MACHINE_IP
2. User - root
3. Password - <ANY>

As you can see the emulated shell is pretty convincing and could catch an unprepared adversary off guard. Most of the commands work like how you'd expect, and the contents of the file system match what would be present on an empty Ubuntu 18.04 installation. However, there are ways to identify this type of Cowrie deployment. For example, it's not possible to execute bash scripts as this is a limitation of low and medium interaction honeypots. It's also possible to identify the default installation as it will mirror a Debian 5 Installation and features a user account named Phil. The default file system also references an outdated CPU.

Answer the questions below

Try running some commands in the honeypot

No answer needed | ✓ Correct Answer

Create a file and then log back in is the file still there? (Yay/Nay)

Nay | ✓ Correct Answer

TASK-4:

Task 4 Cowrie Logs

Cowrie Event Logging

The honeypot wouldn't be of much use without the ability to collect data on the attacks that it's subjected to. Fortunately, Cowrie uses an extensive logging system that tracks every connection and command handled by the system. You can access the real SSH port for this demo machine using the following options:

- IP - MACHINE_IP
- Port - 1400
- User - demo
- Password - demo

Cowrie can log to a variety of different local formats and log parsing suites. In this case, the installation is just using the JSON and text logs. I've installed the JSON parser `jq` on the demo machine to simplify log parsing.

Note: You may need to delete the demo machine's identity from `.ssh/known_hosts` as it will differ from the one used in the honeypot. You will also need to specify a port adding `-p 1400` to the SSH command. The logs will also be found at `/home/cowrie/honeypot/var/log/cowrie`

Log Aggregation

The amount of data collected by honeypots, especially external deployments can quickly exceed the point where it's no longer practical to parse manually. As a result, it's often worth deploying Honeypots alongside a logging platform like the ELK stack. Log aggregation platforms can also provide live monitoring capabilities and alerts. This is particularly beneficial when deploying Honeypots, with the intent to respond to attacks rather than to collect data.

Answer the questions below

Have a look through the logs and see how the activity from the last task has been recorded by the system.

No answer needed — ✓ Correct Answer

TASK-5:

Task 5 Attacks Against SSH

SSH and Brute-Force Attacks

By default, Cowrie will only expose SSH. This means adversaries will only be able to compromise the honeypot by attacking the SSH service. The attack surface presented by a typical SSH installation is limited so most attacks against the service will take the form of brute-force attacks. Defending against these attacks is relatively simple in most cases as they can be defeated by only allowing public-key authentication or by using strong passwords. These attacks should not be completely ignored, as there are simply so many of them that you are pretty much guaranteed to be attacked at some point.

A collection of the 200 most common credentials used against old Cowrie deployments has been left on the demo machine and can be used to answer the questions below. As you can see, most of the passwords are extremely weak. Notable entries include the default credentials used for some devices like Raspberry PIs and the Volumio Jukebox. Various combinations of '1234' and rows of keys are also commonplace.

Answer the questions below

How many passwords include the word "password" or some other variation of it e.g "p@ssw0rd"

15 — ✓ Correct Answer

What is arguably the most common tool for brute-forcing SSH?

hydra — ✓ Correct Answer

What intrusion prevention software framework is commonly used to mitigate SSH brute-force attacks?

Fail2Ban — ✓ Correct Answer

TASK-6:

Task 6 Typical Bot Activity

Typical Post Exploitation Activity

The majority of attacks against typical SSH deployments are automated in some way. As a result, most of the post-exploitation activity that takes place after a bot gains initial access to the honeypot will follow a broad pattern. In general, most bots will perform a combination of the following:

- Perform some reconnaissance using the `uname` or `nproc` commands or by reading the contents of files like `/etc/issue` and `/proc/cpuinfo`. It's possible to change the contents of all these files so the honeypot can pretend to be a server or even an IoT toaster.
- Install malicious software by piping a remote shell script into bash. Often this is performed using `wget` or `curl` though, bots will occasionally use FTP. Cowrie will download each unique occurrence of a file but prevent the scripts from being executed. Most of the scripts tend to reference cryptocurrency mining in some way.
- A more limited number of bots will then perform some anti-forensics tasks by deleting various logs and disabling bash history. This doesn't affect Cowrie since all the actions are logged externally.

Bots are not limited to these actions in any way and there is still some variation in the methods and goals of bots. Run through the questions below to further understand how adversaries typically perform reconnaissance against Linux systems.

Answer the questions below

What's the full model name of CPU does the honeypot "use"?

Intel(R) Core(TM) i9-11900KB CPU @ 3.30GHz — ✓ Correct Answer

Does the honeypot return the correct values when `uname -a` is run? (Yay/Nay)

Nay — ✓ Correct Answer

What flag must be set to pipe `wget` output into bash?

-O — ✓ Correct Answer

How would you disable bash history using `unset`?

unset HISTFILE — ✓ Correct Answer

TASK-7:

Task 7 Identification Techniques

Bot Identification

It is possible to use the data recorded by Cowrie to identify individual bots. The factors that can identify traffic from individual botnets are not always the same. However, some artifacts tend to be consistent across bots including, the IP addresses requested by bots and the specific order of commands. Identifiable messages may also be present in scripts or commands though this is uncommon. Some bots may also use highly identifiable public SSH keys to maintain persistence.

It's also possible to identify bots from the scripts that are downloaded by the honeypot, using the same methods that would be used to identify other malware samples.

Take a look at the samples included with the demo machine and answer the below questions.

Note: Don't run any of the commands found in the samples as you may end up compromising whatever machine that runs them!

Answer the questions below

What brand of device is the bot in the first sample searching for? (BotCommands/Sample1.txt)

Mikrotik — ✓ Correct Answer

What are the commands in the second sample changing? (BotCommands/Sample2.txt)

root password — ✓ Correct Answer

What is the name of the group that runs the botnet in the third sample? (BotCommands/Sample3.txt)

Outlaw — ✓ Correct Answer

TASK-8:

Task 8 SSH Tunnelling

Attacks Performed Using SSH Tunnelling

Some bots will not perform any actions directly against honeypot and instead will leverage a compromised SSH deployment itself. This is accomplished with the use of SSH tunnels. In short, SSH tunnels forward network traffic between nodes via an encrypted tunnel. SSH tunnels can then add an additional layer of secrecy when attacking other targets as third parties are unable to see the contents of packets that are forwarded through the tunnel. Forwarding via SSH tunnels also allows an adversary to hide their true public IP in much the same way a VPN would.

The IP obfuscation can then be used to facilitate schemes that require the use of multiple different public IP addresses like, SEO boosting and spamming. SSH tunnelling may also be used to by-parse IP-based rate limiting tools like Fail2Ban as an adversary is able to transfer to a different IP once they have been blocked.

SSH Tunnelling Data in Cowrie

By default, Cowrie will record all of the SSH tunnelling requests received by the honeypot but, will not forward them on to their destination. This data is of particular importance as it allows for the monitoring and discovery of web attacks, that may not have been found by another honeypot. I've included a couple of samples sort of data that can be recorded from SSH tunnels.

Note: Some elements have been redacted from the samples to protect vulnerable servers.

Answer the questions below

What application is being targetted in the first sample? (Tunnelling/Sample1.txt)

WordPress — ✓ Correct Answer

Is the URL in the second sample malicious? (Tunnelling/Sample2.txt) (Yay/Nay)

Nay — ✓ Correct Answer

TASK-9:

Task 9 Recap and Extra Resources

Recap

I hope this room has demonstrated how interesting honeypots can be and how the data that we can collect from them can be used to gain insight into the operations of botnets and other malicious actors.

Extra Resources

I've included some extra resources to assist in learning more about honeypots below:

- Awesome Honeypots - A curated list of honeypots
- Cowrie - The SSH honeypot used in the demo
- Sending Cowrie Output to ELK - A good example of how to implement live log monitoring

I would also recommend that you deploy a honeypot yourself as it's a great way to learn. Deploying a honeypot is also a great way to **understand** how to work with cloud providers since external honeypots are best when deployed to the cloud. Deploying and managing multiple honeypots is also an interesting challenge and a good way to gain practical experience with tools like Ansible.

Answer the questions below

Read and understand the above

No answer needed — ✓ Correct Answer

RESULT:

Honeypot deployment using Cowrie successfully captured malicious bot activities including SSH brute-force attacks, post-exploitation commands, and SSH tunnelling attempts. Analysis of botnet behavior, attack patterns, credential lists, and bot identification techniques provided valuable insight into real-world attacker methodologies and potential security threats.